

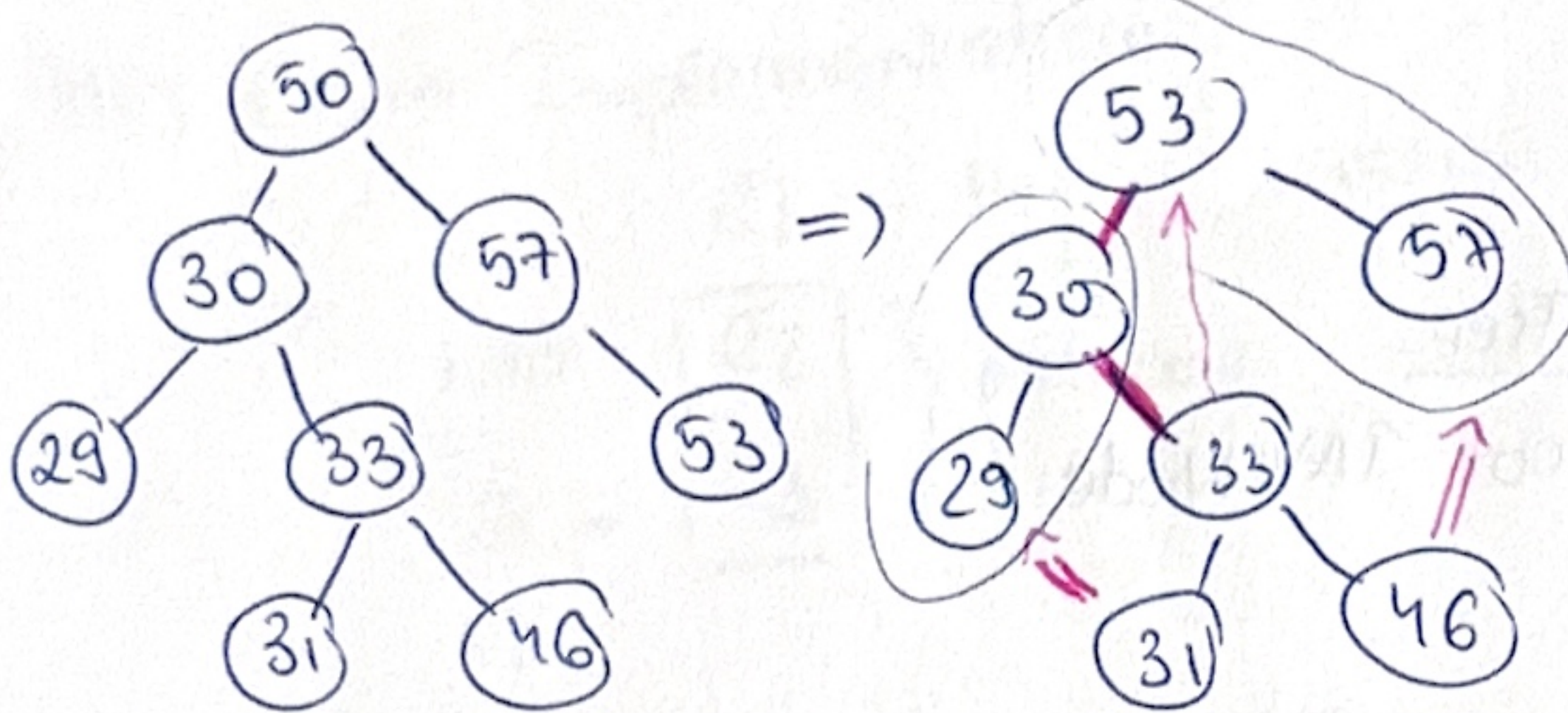
3 June 2026

Lecture 14 - DSA

AVL trees

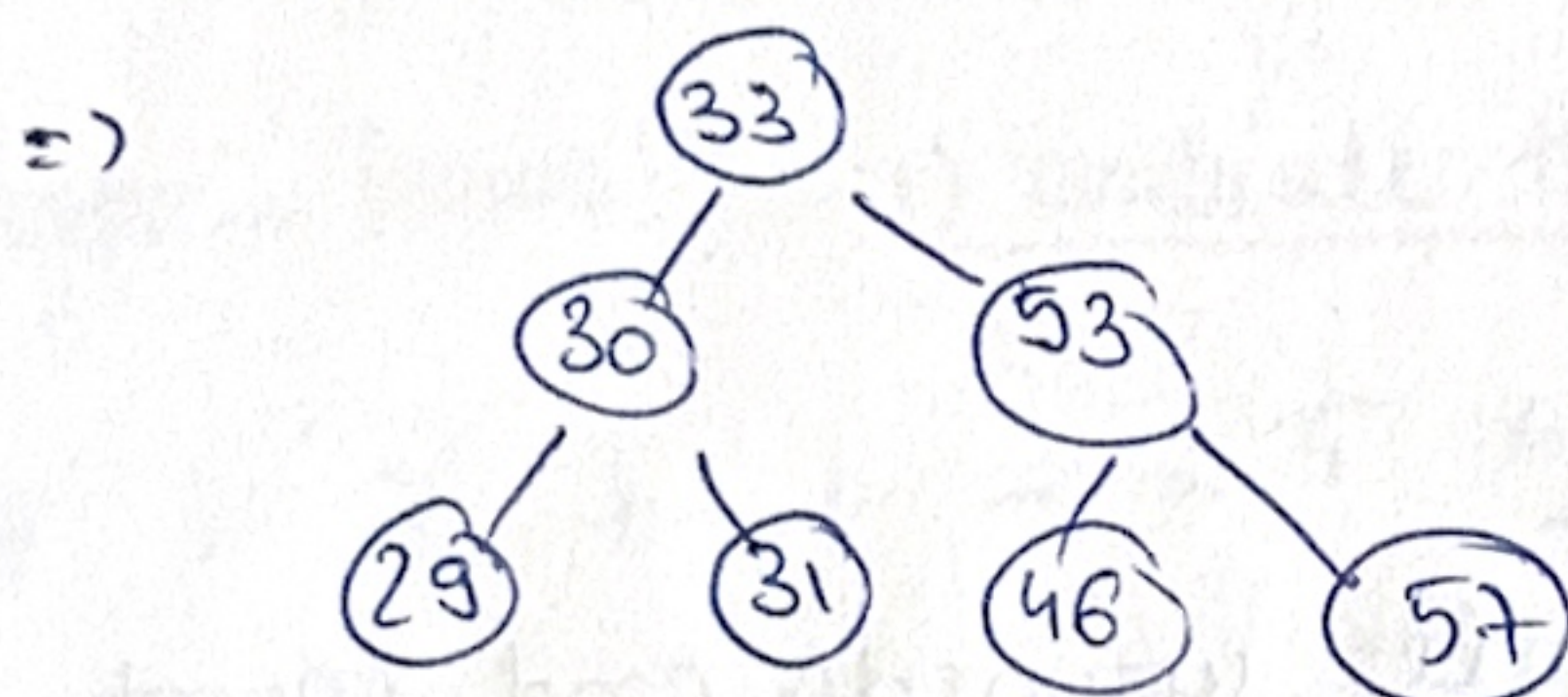
I. REMOVE NODE 50:

- 1) replace 50 with its successor $\Rightarrow$ 53.
- 2) balance the tree.



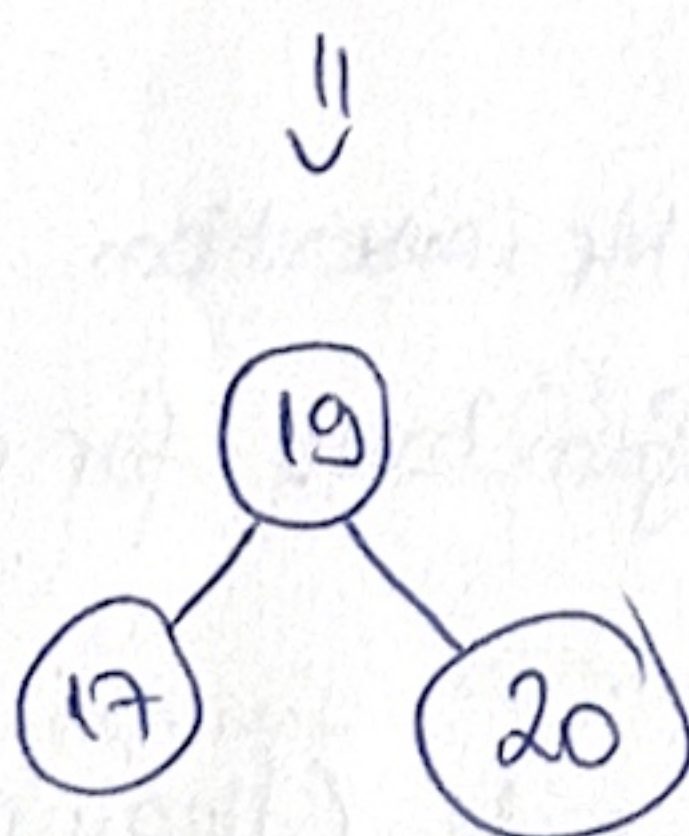
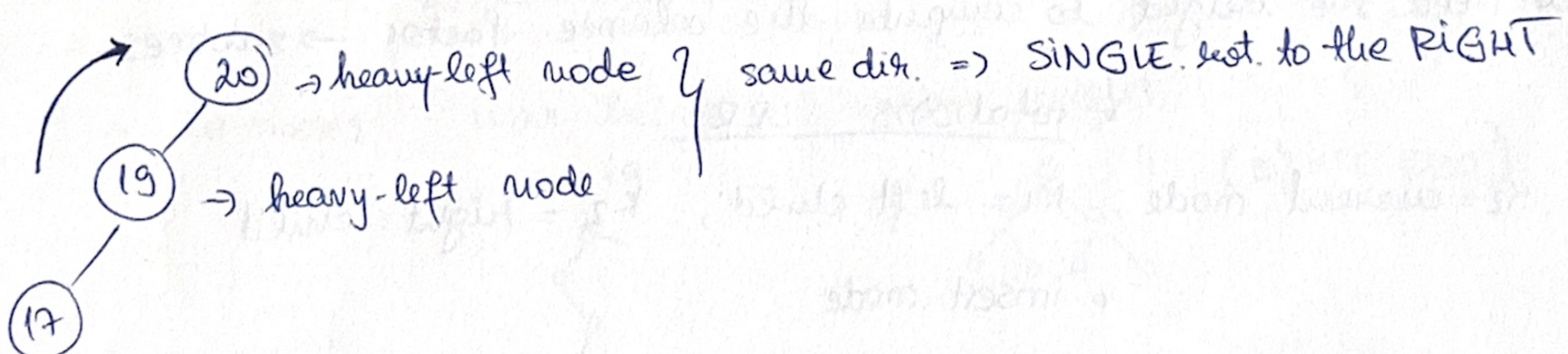
heavy left $\Rightarrow$ rotate right
heavy right $\Rightarrow$ rotate left

33 = root after rotation.
zig-zag $\Rightarrow$ double rotation?



II. REMOVE NODE 25 (root)

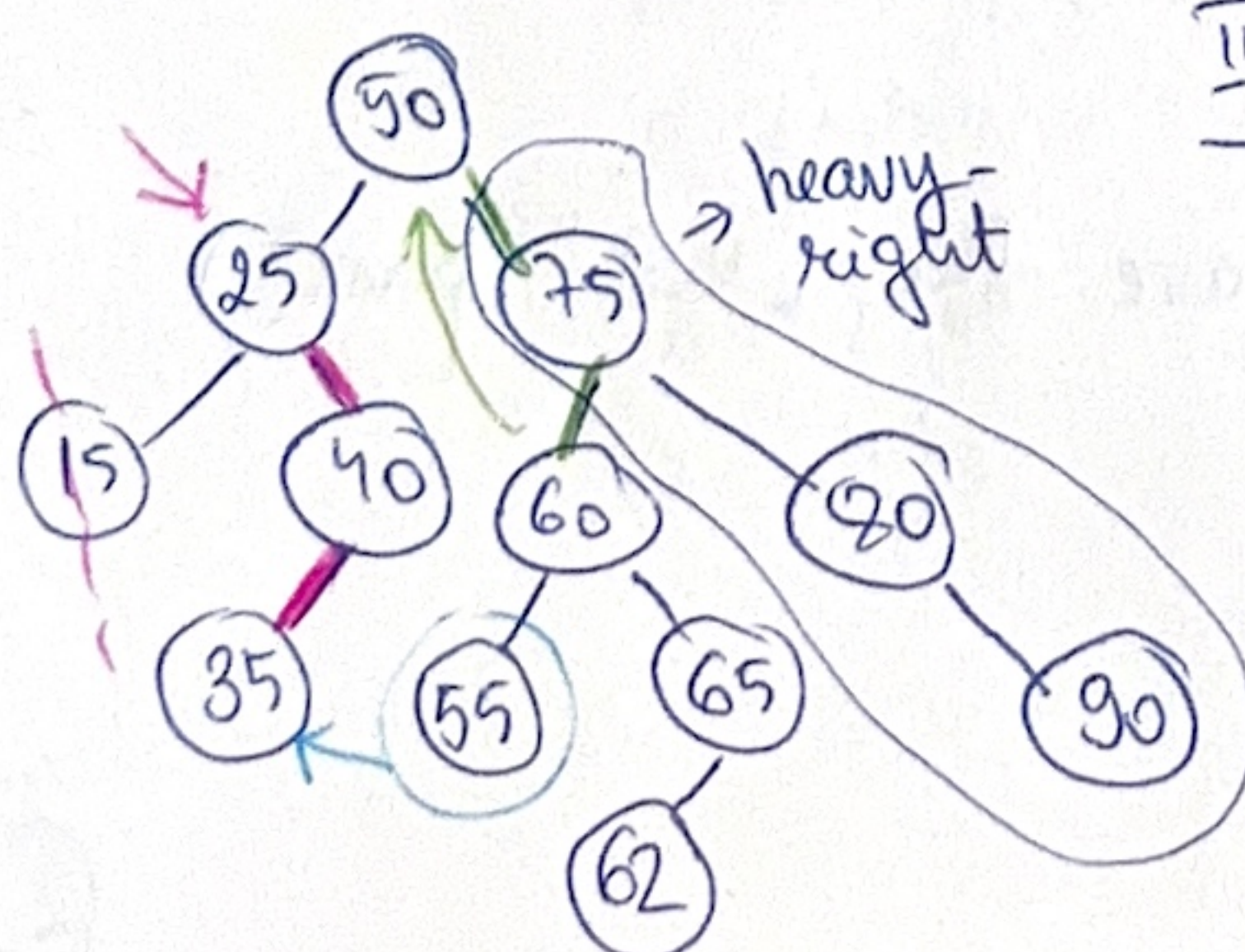
25's predecessor $\Rightarrow$ 21.



20's predecessor $\Rightarrow$ 19. $\Rightarrow$...

when you insert a node $\Rightarrow$ enough one single rotation for it to be balanced.

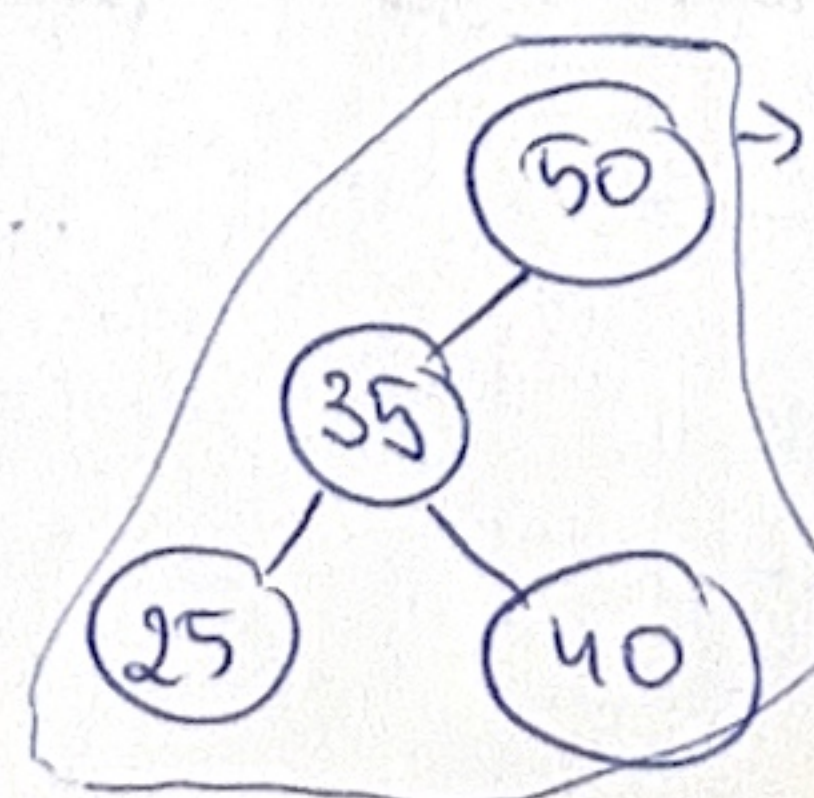
remove



III. REMOVE 15

$\Rightarrow$ DOUBLE rot. to the left

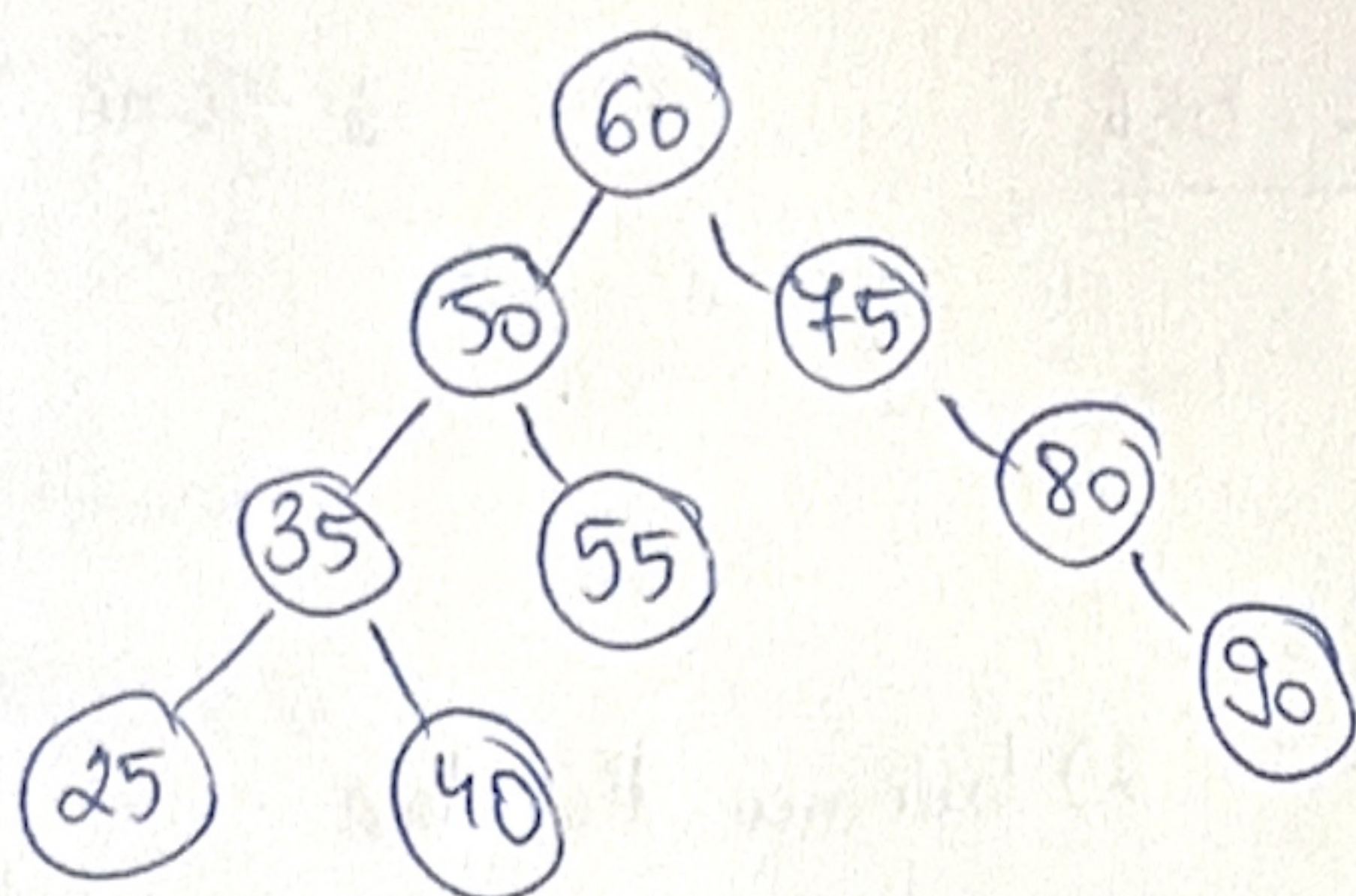
$\Rightarrow$...



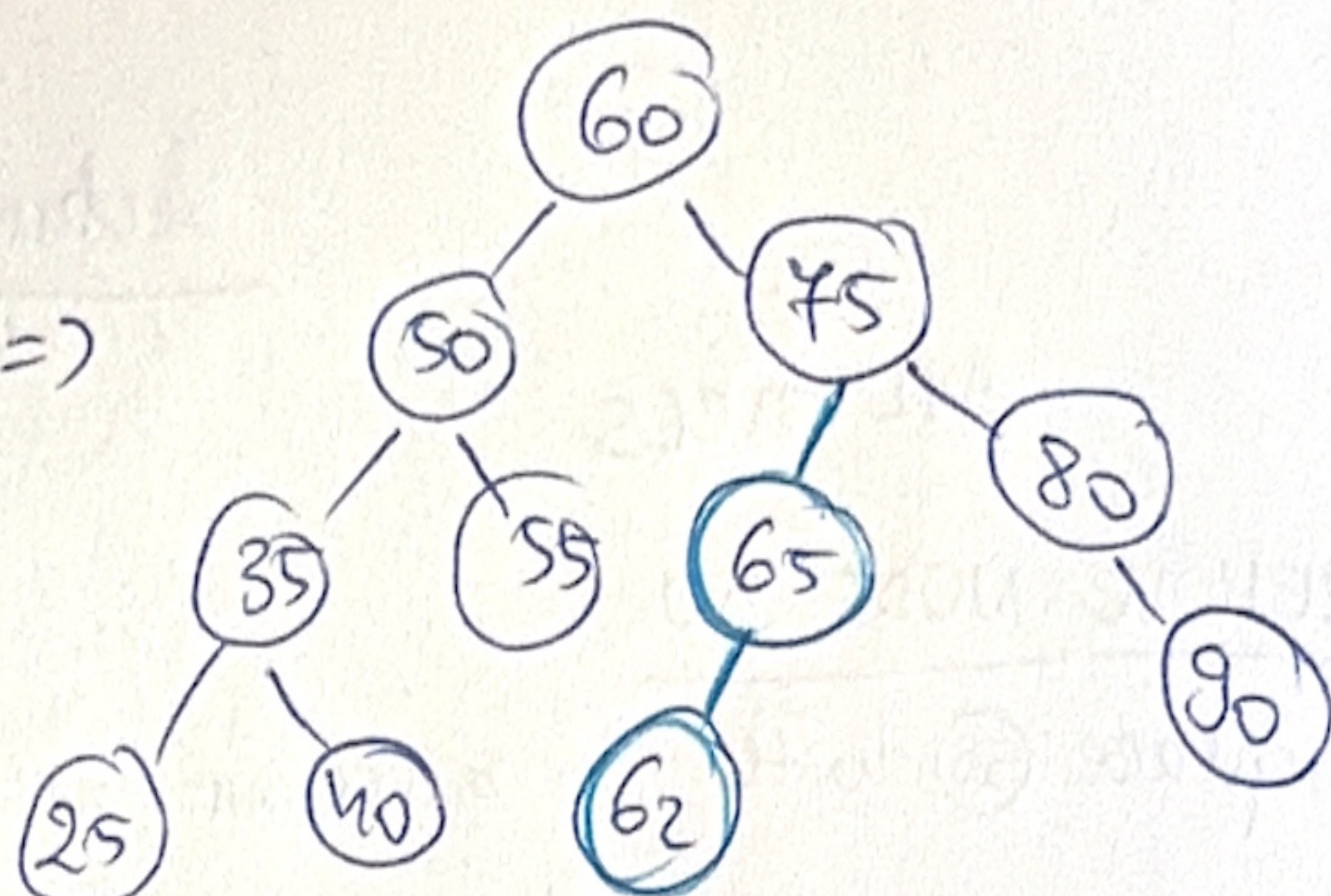
$\Rightarrow$ still imbalanced

$\Rightarrow$ DOUBLE rot. to the LEFT

=>



=>



AVL Representation:

AVLNode:

info: TComp

left: TAVLNode

right: TAVLNode

h: Integer // height of the node

AVLTree:

root: TAVLNode

Implementation for the • height of a tree

→ "height" => UP ↑ => go from leaf nodes upwards and count...?

• balance factor for a node

→ we need the height to compute the balance factor → subtrees...

• rotations: DER

→ K_2 = current node; K_1 = left child; K_3 = right child

• insert node

→ first part => same as BST.

→ recursive method => new root of the subtree after insertion.

=> we need a WRAPPER function for a rec. function

-2 -1 0 1 2

balance = -2 => tree is larger on the right side (heavy-right)

↙

Height of the tree => logarithmic complexity

Example: Student → name, city. What contains?

MultiMap

No pr. language has this implem. (you have the "list" part?)

AP

ADT = containers

DS → used to implement containers.

PART A → 4 multiple choice questions (0.4 p/each) - 20 min, w justification.

PART B → 3 drawing questions - 30 min

PART C → Implementation -

C1 - 14 p. - 15/20 min

1.75 - $\begin{bmatrix} C2 \\ C3 \end{bmatrix}$ } pick one

}

Seminar 6 ⇒ invent new ADT.

Model → Teams!

1) evaluate this expression:

~~4~~ / ~~6~~ / ~~7~~ + 1 - 25 * + +

stack: 4, 6, 7, 13, 1, 12, 2, 5, 10, 22, $\boxed{26}$

→ the stack is enough for justification

2) min/max height

→ 20 nodes

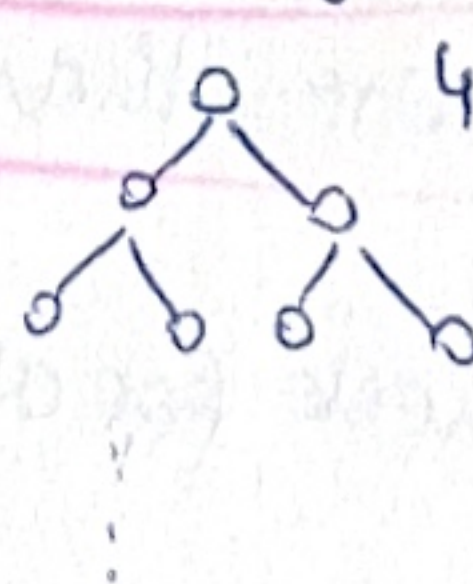
max height

19



min height

4



($2^4 = 16 < 20$)

3) for ADT list → what DS?

→ dynamic array (for singly / doubly → complexity $O(n)$, not efficient.)

4) complexity class? $12m^3 + m \cdot \log m + 52$

$O(m^4)$ ✓

$O(m^2)$

$\Theta(m \cdot \log_2 m)$

$\Omega(m^3)$ ✓

$\Omega(1)$ ✓

$\Theta(m^4)$

leading term: m^3

O : greater than leading term

Ω : smaller than leading term

b) $x'' + 3x' = 3$.

$x_p(t) = At \rightarrow$ first deg. polym.

$x_p'(t) = A$

$x_p''(t) = 0$.

$\Rightarrow 0 + 3A = 3 \Rightarrow A = 1 \Rightarrow x_p(t) = t$.

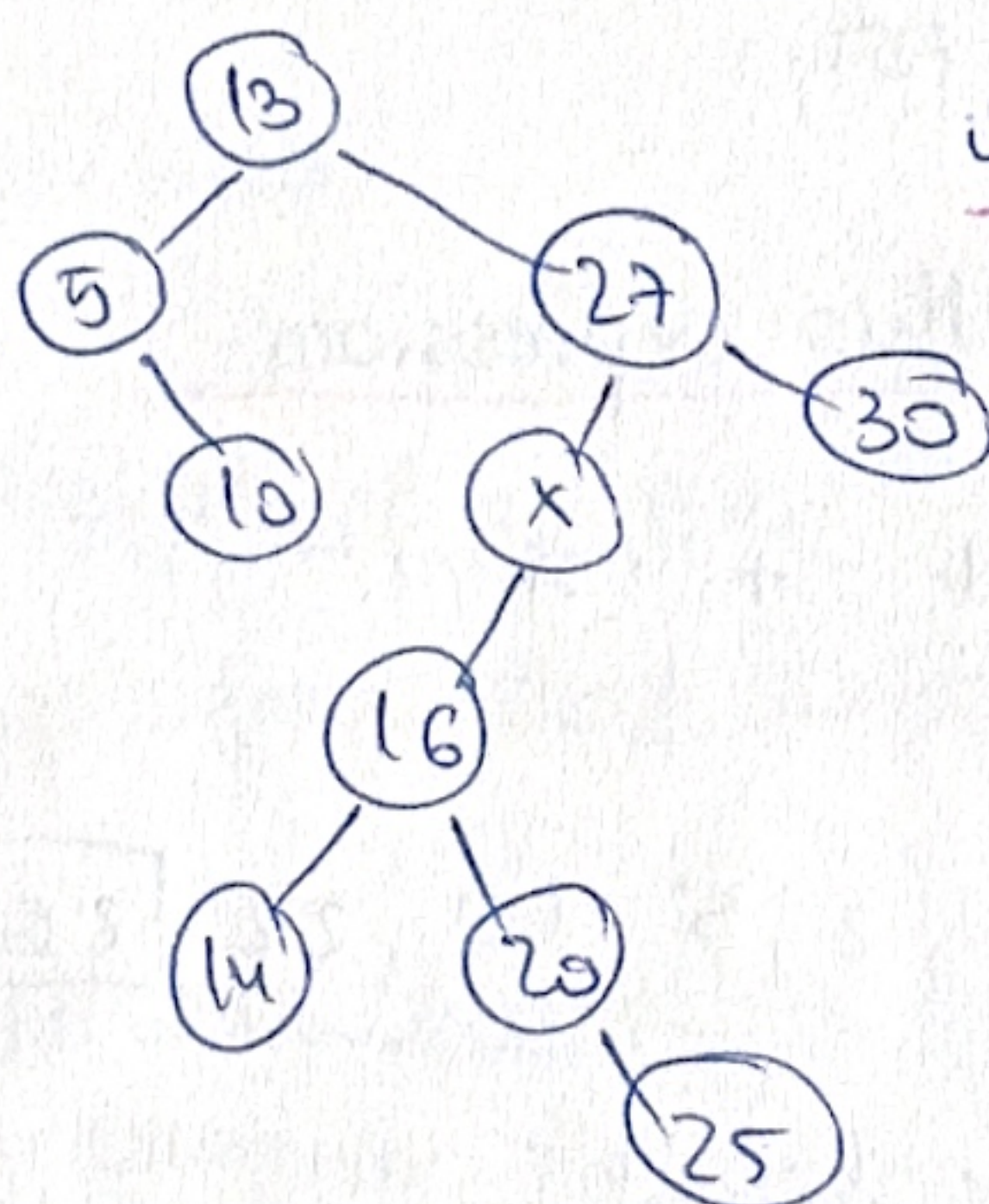
7E

5) how many trees (binomial trees) with 27 elems.

size $\rightarrow$ pw of 2 (sums of pw of 2)

$16 + 8 + 2 + 1 = 2^0 + 2^1 + 2^3 + 2^4 = 27$ elems. $\rightarrow$ 4 trees

6) BST :



what instead of x?

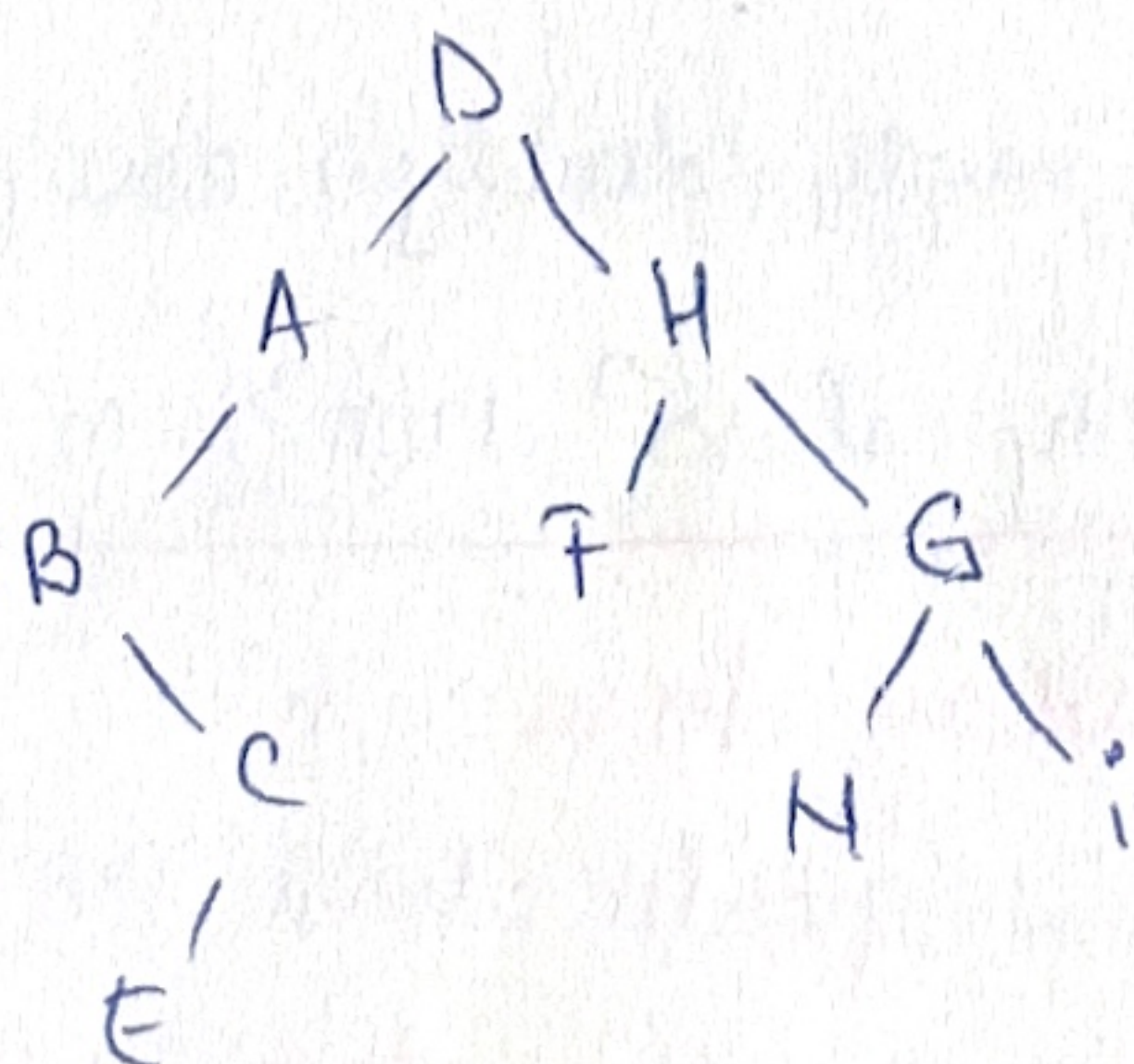
21 } not > 25
24 }
22 }
26 ✓
28

7) hash table; collision res. with open addr; m pos; m elems.

check more than m positions? it is possible

coalesced > check for all
separate

8) Inorder, preorder, postorder, level order :



preorder: D A B C E H F G N I

inorder: B E C A D F H N G I

postorder: E C B A F N I G H D

level-order: D A H B F G C N I E

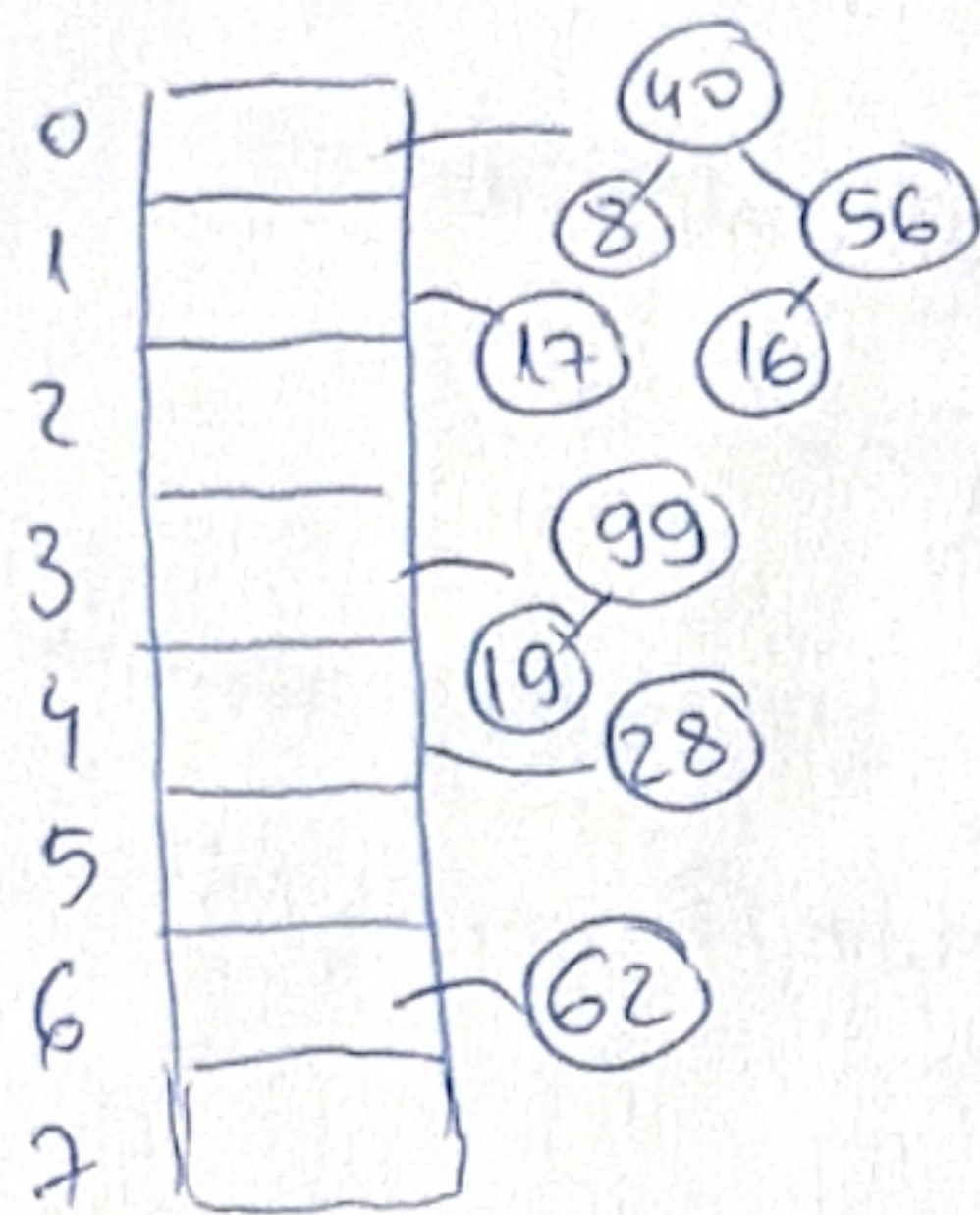
9) HT, $m=8$, division method, AVL instead of a list; add:

8, 99, 40, 19, 56, 16, 17, 28, 62 in HT.

$\rightarrow$ compute value of hash fct. for each elem. $\rightarrow \boxed{e \% m}$

e	8	99	40	19	56	16	17	28	62
h(e)	0	3	0	56	0	0	1	4	6

→ draw hash table, & tree:

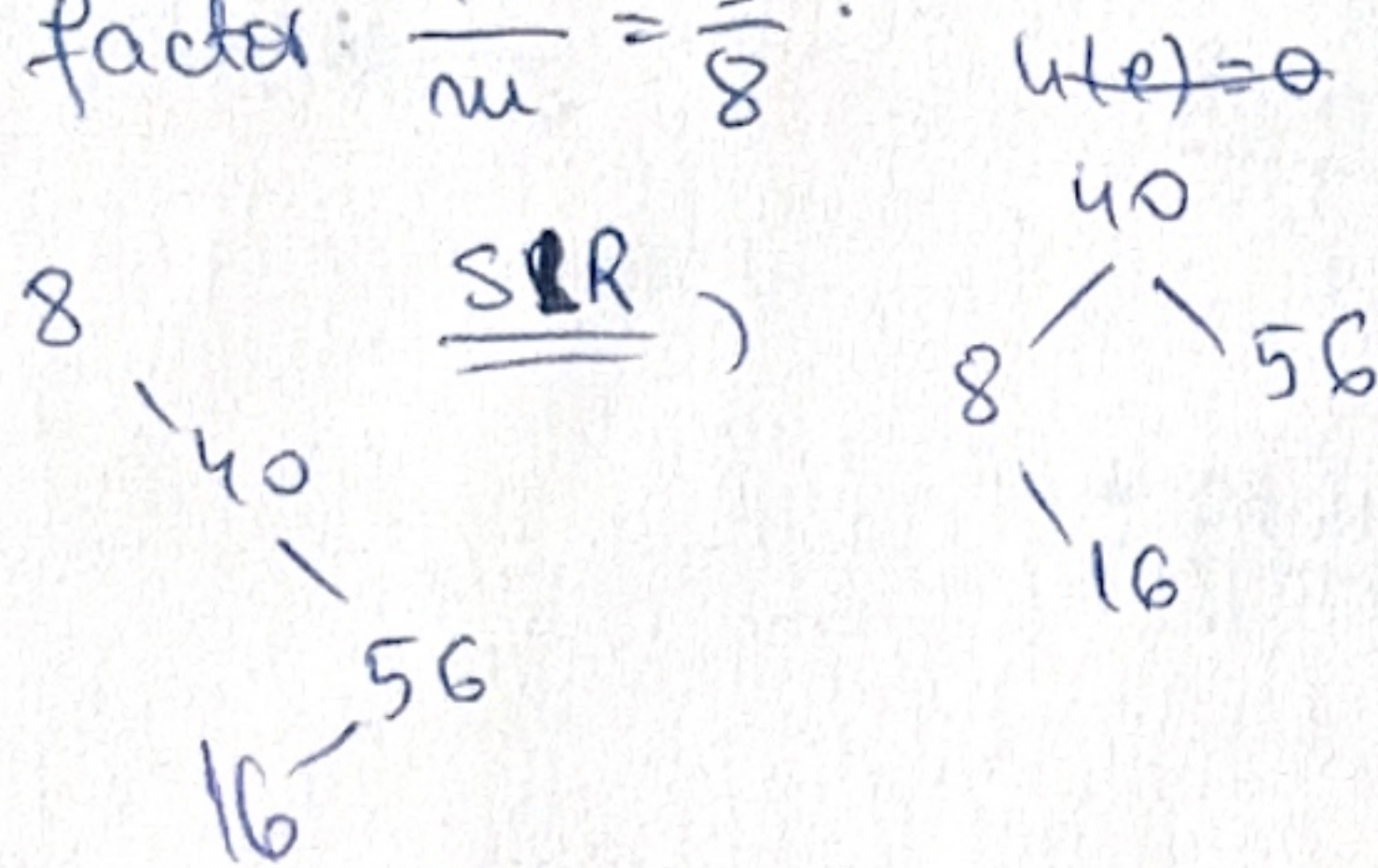


$$\text{load factor} = \frac{m}{n}$$

$m = \text{nr. of nodes}$

$n = \text{size of HT.}$

→ load factor: $\frac{m}{n} = \frac{9}{8}$



ADT Quarbler

init(g) → $\Theta(1)$ ^{total} complexity (TC)

add(g, e) → $O(\log_2 m)$ amortized complexity.

getTopQuarbler(g) → $\Theta(1)$ TC.

deleteTopQuarbler(g) → $O(\log_2 m)$ TC.

? percentile (75% - 25%) → get it to be in $\Theta(1)$

I) start from complexity → what DS have that complexity?

II) start from DS → find which ones match the complexity.

AVL → save that special ... in a separate var. → min → log

Binary Heap

TOP 25% → ?

Bimomial heap, binary heap
↳ no implementation

resize (we rarely resize)

Binary Heap.

